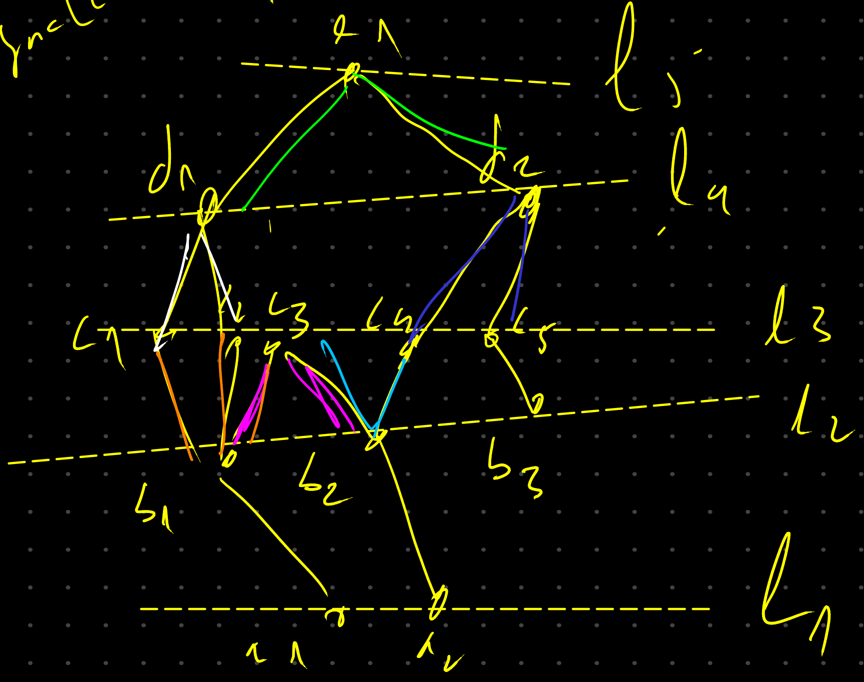
# Compute 2SAT classes

$sync[(c_1,c_2)] = \{(b_1,b_1); (b_1,b_1)\}$




$sync = \{\}$

$sync[(b_1,c_1)] = ((c_1,c_2); (c_1,c_3)) ; ((c_2,c_3))$

$$E = [(a_1,b_1); (a_2,b_2); (b_1,c_1); (b_1,b_2); (b_1,c_3); (b_2,c_3); (b_2,c_4); (b_3,c_5); (c_1,d_1); (c_2,d_1); (c_4,d_2); (c_5,d_2); (d_1,e_1); (d_2,e_1)]$$

d S

$> (a_1,b_1) ; (a_2,b_2)$

first step add all distinct endpoint of adjacent edges to the shared endpoints

$sync[(b_1,c_1)].add((c_1,c_2))$

$sync[(b_1,c_1)].add((c_2,c_1))$

$sync[(c_1, c_2)].add((s_1, s_1))$

$sync[(c_2, c_1)].add((s_1, c_1))$

We loop over sync set-dictionnary

$eq = \{\}$ ; dict;

$(a, b)$ in sync

source    target

$sync \xrightarrow{(sort)}$ sorted sync (by index)

for example $(b_1, b_1)$ $cnt = 1$

$eq[(b_1, c_1)] = set() = eq[(b_1, b_1)]$

$todo = [(b_1, b_1)]$

We get into the todo:

$c, d = todo.pop()$

$b_1$    $b_1$

$$eq[(s_1, s_1)] = eq[(s_1, s_1)]$$

$$eq[(s_1, s_1)].update(sync[(s_1, s_1)])$$

$$sync[(c_1, c_1)] = ((c_1, c_2); (c_1, c_3)); (c_2, c_3)) \longrightarrow eq[(s_1, s_1)] = eq[(b_1, c_1)]$$

$$eq[(s_1, s_1)] = \{(c_1, c_2);\ (c_1, c_3);\ (c_2, c_3)\}$$

$$((c_1, d_1)\ |\ (c_2, d_1))$$

$$todo = \{((c_1, c_2); ((c_1, c_2); (c_2, c_3))\}$$

$$eq[(c_1, c_2)] = eq[(s_1, s_1)]$$

$$eq[(c_1, c_2)].update(\underbrace{sync[(c_1, c_2)]}_{(c_1,\ d_1)})$$

$\varsigma[(l_1, l_n)] \leftarrow (c_1, c_2)$

$\varsigma[(l_1, l_n)] \leftarrow (c_2, c_1)$

$\varsigma[(c_1, c_2)] \leftarrow (d_1, k_2)$

$\varsigma[(c_2, c_1)] \leftarrow (d_1, k_1)$

$\varsigma_{nc}[(c_1, c_2)] = \{(b_1, b_1) ; \underbrace{(l_1, l_1)}\}$

$eq[(d_1, l_1)] = eq[(b_1, b_1)] = eq[(c_1, c_2)]$

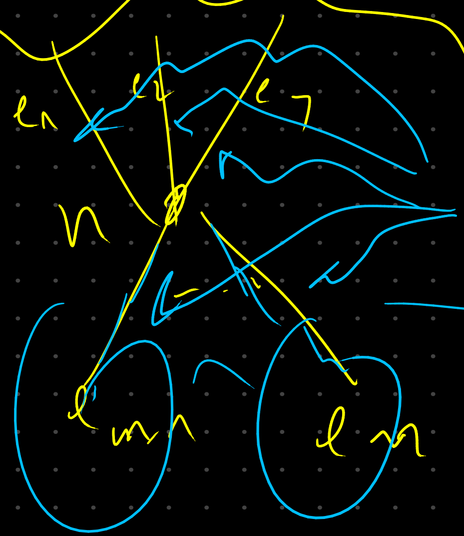
buildlevelgraph(ems)

$\{(y_1, lvl_1), (y_2, lvl_1), \dots$

$(x_1, h_1) \; (x_2, h_2) \dots (x_n, h_n)$

for i, nodes in enumerate(ends):
    for n in nodes
        hasInEdges = false

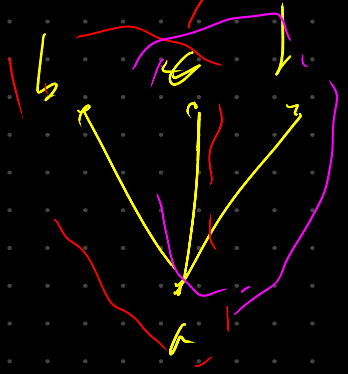
        for adj in n.adjEntities:
            if (!adj -> isSource())
                hasInEdges = true




n.adjEntities()

adj.isSource() == False

if !hasInEdges
    warning.add{n}




sync[(a,a)].add((b,c)) {(b,c);
(c,d);
(b,d)}

sync[(a,a)].add((c,d))

sync[(a,a)].add((b,d))

todo = [(a,a)]

eq[(a,a)] = eq[(a,a)]   eq[(a,a)].update(sync[(a,a)])

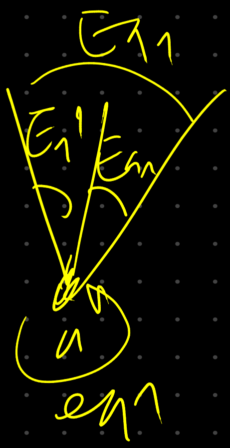
todo = [(b,c); (c,d); (b,d)]   {(b,c); (c,d); (b,d)}

missing

eq[(b,c)] = eq[(a,a)] = [(b,c); (c,d); (b,d)]

sync[(b,c)] = sync[(b,b)] ⊔ (a,c)

⇒ eq[(b,c)] = eq[(a,b)] − [(b,c); (c,d); (b,d)]
(c,d)



We solve the transitivity clause issue when it comes to the order of distinct vertices same level like adjacent edges to the same shared endpoint.

* second issue :

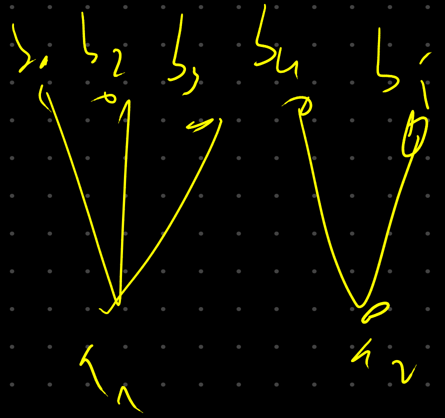
HashMap Adj. entry (that is source)

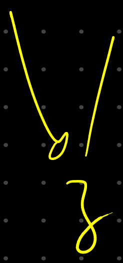
All the adj. edges (vertices) should have the same Eq class to the questionable vertex

1. detect troublesome vertices
2) criteria to protect.

```
AdjSource[u].add(x)
            .add(y)

if (!v.hasIncomingEdge()) { warning.add(v) }
```





$sync[(a_1, a_1)] = [(b_1, b_2), (b_1, b_3), (b_2, b_3)]$

$sync[(a_2, a_2)] = [(b_4, b_5)]$

Loop over all vertices of $L_1$ ($x$)
Loop over AdjSource[$x$] ($v$)
Loop over warnings ($w$)

$sync[(v, w)] = sync[(v_0, w_0)]$

(We need to minimize it with rel. order) (check if we already have a guide)